UNIVERSIDADE D
COIMBRA

Operative Systems- Bachelor's Degree In Informatics Engineering

DEIChain: A Concurrency-Focused Blockchain Simulation

Project by:

Diogo André de Freitas Alves – 2020214197

Bruno Miguel Santos Marques – 2018278019

Table of Contents

System Components

1.1 – blockchain_ledger.c	static void lock(); static void unlock();	The lock() performs a "down" (P) operation on the ledger's semaphore (sem_id) to enter a critical section, ensuring exclusive access to the shared ledger data. The unlock() performs an "up" (V) operation on the ledger's semaphore to leave the critical section, allowing other processes to access the shared ledger	
void init_ledger();	Creates or attaches to the shared-memory segment that holds the blockchain ledger structure.	void add_block (Block *block);	Appends a newly mined block to the ledger.
int get_block_count();	Returns the current number of blocks stored in the ledger.	void print_ledger();	Displays the entire contents of the ledger on standard output.
void cleanup_ledger();	Detaches from and removes the shared-memory segment.	SharedLedger *get_ledger();	Returns the (process-local) pointer to the shared SharedLedger structure for direct read or manipulation by other modules (without re-attaching).
1.2 – cleanup.c			
void cleanup_shared_memory (int shm_id);	Marks the System V shared-memory segment identified by shm_id for removal (IPC_RMID).	void cleanup_message_queue (int msg_id);	Marks the System V message queue identified by msg_id for removal (IPC_RMID).
void cleanup_fifo (const char *fifo_path);	Removes (unlinks) the named pipe (FIFO) at the given filesystem path.		
1.3 – controller.c			
void create_and_hold_fifos();	For each of the three Validator slots, a unique FIFO path *(/tmp/VALIDATOR_INPUT_i)* is made, removes any existing FIFO at that path, creates the FIFO, and opens it read/write to keep the endpoint alive even if no writer is connected.	void handle_sigusr1(int sig);	Triggered when the Controller receives a SIGUSR1 signal. It reads the current contents of the blockchain ledger and prints/dumps them both to the screen and to the DEIChain_log.log file.
void cleanup(int sig);	Handles graceful shutdown when the program receives a SIGINT (Ctrl+C).	pid_t create_process(const char *name, const char *path, const char *arg);	Forks a new process and execl()s the specified path.Returns the child PID in the parent, or never returns in the child (exits on failure).
int get_txpool_occupancy();	Calculates how full the transaction pool is as a percentage.	void adjust_validators();	Dynamically scales the number of validator processes based on transaction pool load.
int process_alive(pid_t pid);	Checks whether a process with the given PID is currently alive by sending signal 0. Returns nonzero if the process exists and the caller has permission to signal it.		
1.4 – logger.c			
int init_logger();	Opens (or creates) the log file *"DEIChain_log.log"* in append mode and stores its *FILE** in a static variable.	void close_logger();	Safely closes the log file if it is open. Uses a mutex to ensure no other thread is writing at the same time.
void log_message(const char *format, ...);	Generates a timestamp string. Writes the formatted message (using printf-style formatting) both to **standard output** and to the **log file**, each prefixed with the timestamp. Uses a mutex to synchronize access, preventing interleaved messages when multiple threads or processes log concurrently.		
1.5 – miner.c			
void sha256(const char *str, char output[65]);	Computes the SHA-256 hash of the null-terminated string str.	int check_difficulty(char *hash);	Verifies whether the given hex hash string has at least DIFFICULTY leading zeros.
void *mine_block(void *arg);	Main thread routine for mining blocks.	void setup_fifo_names();	Populates fifo_names[] with the three FIFO paths *(VALIDATOR_INPUT_0, _1, _2)* used to broadcast mined blocks to each Validator.
void handle_sigint(int sig);	Signal handler for SIGINT that sets a stop_flag, causing all mining threads to exit gracefully and log receipt of the signal.	void broadcast_block(Block *block);	Iterates over each of the three FIFO paths, opens it for writing, acquires an exclusive file lock, writes the Block structure into the pipe, releases the lock, and closes the descriptor—ensuring each Validator receives the new block
1.6 – statistics.c			
void handle_sigusr1(int sig);	Signal handler for **SIGUSR1** that simply invokes print_statistics(), allowing on-demand reporting without terminating the process.	void handle_sigint(int sig);	Signal handler for **SIGINT** that logs receipt of the signal, calls print_statistics(), closes the log, and exits cleanly.
void print_statistics();	Logs a "Statistics Summary" header, then iterates over all miners (up to MAX_MINERS) and prints each miner's count of valid and invalid blocks plus accumulated credits (only if they have at least one block). Finally, prints the overall total_blocks. Flushes stdout to ensure immediate output.		
1.7 – transaction_pool.c			
void lock_transaction_pool();	Performs a "down" (P) operation on the pool semaphore (sem_id) to enter a critical section, preventing concurrent modifications to the shared transaction array.	void unlock_transaction_pool();	Performs an "up" (V) operation on the pool semaphore to leave the critical section, allowing other processes or threads to modify the pool.
void init_transaction_pool();	Validates TX_POOL_SIZE, allocates or attaches to the shared-memory segment, resets it if corrupted, and creates two semaphores (one mutex, one counting).	int add_transaction_struct(Transaction *tx);	Wrapper that unpacks a Transaction struct's fields and calls the full insertion routine.
int add_transaction_full(int id, int reward, float value, time_t timestamp, const char *data);	Locks the pool, rejects duplicates or overflow, inserts a new transaction, signals miners when a block is ready, unlocks, and returns success or failure.	void remove_transactions(int num);	Locks the pool, shifts remaining transactions to remove the first num, updates the count, logs the removal, and unlocks.
Transaction *fetch_transactions(int *num);	Locks the pool, sets *num to up to TRANSACTIONS_PER_BLOCK, returns a pointer to the array, and unlocks.	void wait_for_transactions();	Blocks on the counting semaphore until enough transactions accumulate to form at least one block.
void attach_transaction_pool();	In non-creator processes, attaches to the existing shared-memory pool and its semaphores.	void detach_transaction_pool();	Calls detach_transaction_pool(), removes the shared-memory segment and both semaphores, and logs cleanup
SharedTransactionPool *get_transaction_pool(void);	Returns the local pointer to the shared pool for read-only inspection.		
1.8 – transaction_generator.c			
Transaction create_transaction(float value, const char *data);	Atomically assigns a unique ID, sets the reward to 1 and age to 0, timestamps the transaction, copies the provided data string, and returns the initialized Transaction struct.	int generate_and_add_transaction(float value, const char *data);	Calls create_transaction() to build a new transaction, then attempts to insert it into the shared pool by calling add_transaction_full(); returns 1 if the insertion succeeds or 0 on failure.
1.9 – txgen.c			
void cleanup_txgen();	Detaches from the shared transaction pool and closes the logger, ensuring that IPC resources and the log file are properly released when the TxGen process exits.	void handle_sigint(int sig);	Installed as the SIGINT handler, this function logs receipt of the signal, calls cleanup_txgen() to free resources, and then calls exit(0) to terminate the process cleanly.
1.10 – validator.c			
void handle_sigint(int sig);	Catches **SIGINT**, logs it, closes the FIFO, detaches from the transaction pool, closes the logger, and exits.	int validate_pow(const char *hash);	Returns 1 if the first DIFFICULTY characters of hash are '0', otherwise returns 0.
void init_validator_semaphore();	Creates or retrieves the System V semaphore (SEM_KEY+2) used to serialize block validation.	void lock_validator();	Performs a "down" (P) operation on the validator semaphore to enter the critical section before validating a block.
void unlock_validator();	Performs an "up" (V) operation on the validator semaphore to leave the critical section after validating a block.	void age_transactions();	Locks the transaction pool, increments each pending transaction's age, and boosts its reward every time age % 50 == 0, then unlocks the pool.
void remove_block_transactions(const Transaction *block_tx);	Locks the pool, locates and removes each transaction whose ID matches one in block_tx, shifts the remaining entries, decrements the count, logs the result, and unlocks.	int open_fifo_for_reading();	Repeatedly opens the configured FIFO for reading, logging each retry and sleeping between attempts until it succeeds, then returns the file descriptor.
int read_block_serialized(int fd, Block *block);	Reads exactly sizeof(Block) bytes from fd into block, handling partial reads, closed writers (by reopening), and transient errors (EINTR, EAGAIN), returning 1 on success or 0 on a fatal error.	int validate_block(Block *block, int *total_credits);	Applies aging, checks previous_hash against the last ledger entry (or "INITIAL_HASH" for the first block), sums transaction rewards into *total_credits, verifies the PoW, removes those transactions from the pool, appends the block to the ledger if valid, and returns 1 for valid or 0 for invalid.

Summary and Main Features

2.1 – Initialization

When the simulation starts, the Controller reads config.cfg to set the number of miners, pool size, transactions per block, and blockchain length. It then opens the log and creates all IPC resources: two shared-memory segments (one for the Transaction Pool and one for the Blockchain Ledger), a semaphore to guard pool access, a message queue for validation results, and a FIFO at /tmp/VALIDATOR_INPUT. At this point the Controller also prepares for the **aging mechanism** by ensuring the Validator will be able to increment each transaction's age and boost its reward every 50 age units. Finally, the Controller forks and execs the Miner process, an initial Validator, and the Statistics process, leaving the system ready to operate.

2.2 – Controller

Every two seconds, the Controller attaches briefly to the Transaction Pool to measure its occupancy and dynamically adjusts the number of Validator processes between one and three—adding when occupancy exceeds 60 % and 80 %, removing extras when it falls below 40 %. It also handles **SIGINT**, upon which it logs the shutdown, signals all child processes to exit, removes all IPC resources (shared memories, semaphore, message queue, FIFO), closes the log, and then terminates itself..

2.3 – Transaction Generator

Each TxGen process takes two command-line parameters—reward (1–3) and sleep_time (200–3000 ms). After initializing its logger and attaching to the shared pool, it loops until it has successfully inserted TX_POOL_SIZE transactions. In each iteration it uses create_transaction() to build a new transaction (atomic ID, random value, age = 0, and descriptive data), then sets its reward, and calls add_transaction_struct(). On success it increments its local counter; on failure (such as a full pool), it logs the retry, backs off 50 ms (logging every 10 futile attempts), and tries again. Between attempts it sleeps for the specified interval. On SIGINT or when the counter reaches TX_POOL_SIZE, it detaches from the pool, closes the log, and exits cleanly.

2.4 – Miner

The Miner process initializes pool access and logging, then spawns NUM_MINERS threads. Each thread repeatedly fetches up to TRANSACTIONS_PER_BLOCK pending transactions (including their age values), builds a block, and performs a simplified Proof-of-Work by iterating nonces and hashing until the hash meets the DIFFICULTY requirement. Once a valid nonce is found, the thread writes the block to the FIFO, logs the start and end of mining, removes those transactions from the pool, and sleeps briefly. Any I/O errors on the FIFO cause the thread to log the error and exit.

2.5 – Statistics

The Statistics process opens its log, installs a **SIGUSR1** handler, and attaches to the message queue. It enters a loop, receiving messages from Validators containing (miner_id, credits, valid_flag). It updates per-miner arrays—valid_blocks[], invalid_blocks[], and credits[]—and a global total_blocks counter. When **SIGUSR1** arrives, it prints a snapshot showing each miner's valid/invalid block counts and credits, followed by the overall block count.

2.6 – Cleanup

Upon **SIGINT**, the Controller logs receipt and sends termination signals to all Miner threads, all Validator processes, and the Statistics process. Each child then detaches from shared memory, unlinks the FIFO if applicable, closes its log, and exits. Finally, the Controller removes every IPC resource (shared memories, semaphore, message queue, FIFO), closes its log, and exits, ensuring a completely clean shutdown.

Note: This is highly Summarized due to space constraints.

Time spent on the project: 12h – Diogo André de Freitas Alves; 6h – Bruno Miguel Santos Marques